AQoSE

Adaptive QoS Engine

A predictive traffic classifier that tells Linux CAKE
which connection is the call or the game

Full project report

Tata Elxsi Teliport, Season 4
Team: Shreyash Neeraj, Mridul Jaiswal, Sachi Jha
Original study: October 16, 2026
Rust runtime update: October 22, 2026

Source code, data pipeline, results and this report:
`github.com/shreyashsri79/aqose`

Project explanation video:
`youtu.be/k6ccbBaxwUg`

The network results were measured in a rootless network-namespace testbed on one laptop (Fedora 43, Linux 7.2.8, 12th-gen Intel Core i5, 16 threads). The Rust update adds component benchmarks and a six-hour stability run on the same host. The system has not yet run on a real router or in a real home. The repository is `https://github.com/shreyashsri79/aqose`; local evidence paths for the runtime update are listed in Section 5.12.

Summary

Home routers that run CAKE, the strongest open-source queue manager, already keep most calls and games smooth: CAKE shares the line fairly between devices and then between each device's connections. It fails in one common situation: *a call or a game on a device that is also downloading, uploading or sharing files.* Fairness then gives the call a small slice of its own device's share, and nothing tells CAKE which connection is the call. Existing tools sort traffic with fixed thresholds ("many packets per second means bulk"), which mistake video calls and cloud games for downloads because they also send many packets.

AQoSE is an add-on to CAKE. It watches the sizes and timing of the first eight packets of each new connection (headers only, never content), predicts the kind of traffic with a small gradient-boosted tree model, picks the CAKE priority tin with the lowest expected damage, and writes that choice as a connection label CAKE reads for every packet. A safety shield checks every change, a verifier undoes changes that raise delay, and a separate watchdog returns the router to plain CAKE within about two seconds if AQoSE stops.

Late video or game frames (median of 10 seeds)	Plain CAKE	cake-autorate	Threshold rules	**AQoSE**
S6 HD call on a laptop that is sharing files	94.3%	–	94.4%	**0.6%**
S5 work call on a busy evening, backup running	93.3%	93.3%	78.7%	**0.1%**
S7 cloud game on a PC fetching an update	83.2%	–	19.4%	**1.0%**
S2 line drops to a fifth of its speed	55.5%	4.6%	55.6%	**0.9%**
S12 a real WhatsApp video call, replayed (5 seeds)	57.0%	–	42.7%	**2.4%**

- **Against plain CAKE**, every seed-paired run of the frozen system: better in 49, tied in 41, worse in 0 on seeds 0–9, and 21, 29, 0 on fresh seeds never used in development.
- **Against cake-autorate**, the community tool for changing line speed, better in all 20 runs: 0.9% late frames against its 4.6% when the line slows (S2), and 0.1% against 93.3% on a call sharing its device with a backup (S5), which no rate setting can fix.
- **Real traffic, replayed.** Two recorded WhatsApp calls, replayed in the emulator, were recognised within 0.13 s in all 20 runs; late frames 57% → 2.4% (a call the model never trained on) and 79% → 14% (a larger one it trained on).
- **Sealed test set**, opened once: accuracy 0.969 but macro-F1 0.73 on 1,213 flows from 183 unseen sessions, with only 4–8 flows in some classes; no video call or cloud game sent to Bulk, partly by design, as UDP is never demoted (qosify's defaults: 14% and 78%).
- **Against the threshold rules** existing tools use, never worse in any run (52 better, 38 tied). Over the nine scenarios they leave 6.61 flows unusable, the classifier alone 0.70, the full system 0.69: the improvement comes from the classifier.
- **No starvation, fairness kept.** Bulk transfers keep a median 98% of their plain-CAKE throughput (75–104% by scenario) and never starve more than under plain CAKE; Jain's fairness index between devices is unchanged within ± 0.01.
- **Jitter, loss, link estimation.** A call's packet loss stayed at most 0.03% where plain CAKE lost up to 91%. After the line drops, the estimator is under the new rate within 2 s, with 6% steady-state error (cake-autorate: 20%). IPv6 end to end (S14): late frames 94.3% → 0.3%.
- **Rust runtime update.** Separate matched benchmarks measured 52% lower median classifier request latency, 53% lower classifier resident memory and 43% less controller CPU in short runs. A six-hour Rust stability run passed. These measurements supplement the original study; the call, game, accuracy and throughput results above have not been rerun as a full Rust matrix (Section 5.12).

The defensible claim is narrower than "beats CAKE" and sharper: **In emulation, AQoSE beats plain CAKE when a call or a game shares its device with heavy traffic, and when the line's speed**

changes; elsewhere it ties CAKE, and it never did worse than threshold rules. It has not yet run on a real router or in a real home. Section 8 lists the limits; everything is reproducible from the public repository, `github.com/shreyashsri79/aqose`. A video explaining the project is at `youtu.be/k6ccbBaxwUg`.

Contents

Summary **1**

1 The problem **5**

2 Design **6**

2.1 What happens, step by step . . . 6
2.2 System architecture . . . 7
2.3 Layers . . . 7
2.4 Principle: no AQoSE code on the per-packet path . . . 8
2.5 Components . . . 9
2.6 Module contracts . . . 13
2.7 The safety net . . . 14
2.8 How the line is shared between the four tins . . . 15

3 Data and model **17**

3.1 Dataset . . . 17
3.2 Model choice: LightGBM over a random forest . . . 17
3.3 Offline results . . . 17

4 How it was tested **18**

4.1 Testbed . . . 18
4.2 Traffic and measurement . . . 18
4.3 Scenarios and configurations . . . 18
4.4 Protocol . . . 18

5 Results **19**

5.1 The frozen system, seeds 0–9 . . . 19
5.2 Fresh seeds and an unseen scenario . . . 20
5.3 Real calls, replayed . . . 20
5.4 Is expected damage the reason? . . . 21
5.5 Bulk progress and fairness . . . 21
5.6 Jitter, loss and throughput . . . 22
5.7 Link estimation . . . 22
5.8 Policy stability and adaptation speed . . . 23
5.9 IPv6 end to end . . . 23
5.10 Dashboard . . . 23
5.11 Cost . . . 24
5.12 Rust runtime efficiency and stability update . . . 24

6 From prototype to product **26**

7 What we learned **27**

8 Limitations **27**

9 Next steps: from emulation to real homes **28**

10 Conclusion **29**

A Reproducing the results **29**

1 The problem

What CAKE already does well. CAKE (Common Applications Kept Enhanced) shapes traffic just below the line rate so that queues build in the router, where they can be managed, and not in the modem's oversized buffer. It shares the link fairly between hosts, then between each host's flows. In our testbed this alone takes a call's late frames from 77% (no queue management) to 0% whenever the call is alone on its device (scenarios S1, S4, S10).

Where it fails. Fairness protects a call from *other* devices, not from its own. A laptop on an HD video call that is also running a file-sharing client with twenty connections gives the call one twenty-first of its share. In scenario S6 that leaves 94% of the call's frames late. The same happens to a cloud game on a PC fetching a game update (S7, 83% late).

Why tins are not used today. CAKE's `diffserv4` mode has four priority tins: Voice, Video, Best Effort and Bulk. Something must decide which flow belongs in which. Applications rarely set priority marks that survive the internet, and the tools that sort traffic on home routers (qosify on OpenWrt, for example) use thresholds on packet rate and size. An HD video call sends 400–600 packets a second and a cloud game over 1,000, so those rules call both "bulk". On our sealed test set, qosify's default rules sent 78% of cloud-gaming flows and 14% of video-call flows to Bulk.

2 Design

2.1 What happens, step by step

Figure 1 follows an ordinary evening: a video call starts on a laptop, and the same laptop then starts a cloud backup. The router recognises each connection from its first eight packets and puts the call in the Video tin and the backup in Bulk (or Best Effort when it is not sure). After that, a cheap check each second compares each connection's traffic with its class's normal range; the classifier is asked again only when a connection stays outside that range for 2 to 6 seconds, or at a routine recheck every 30 to 60 seconds.

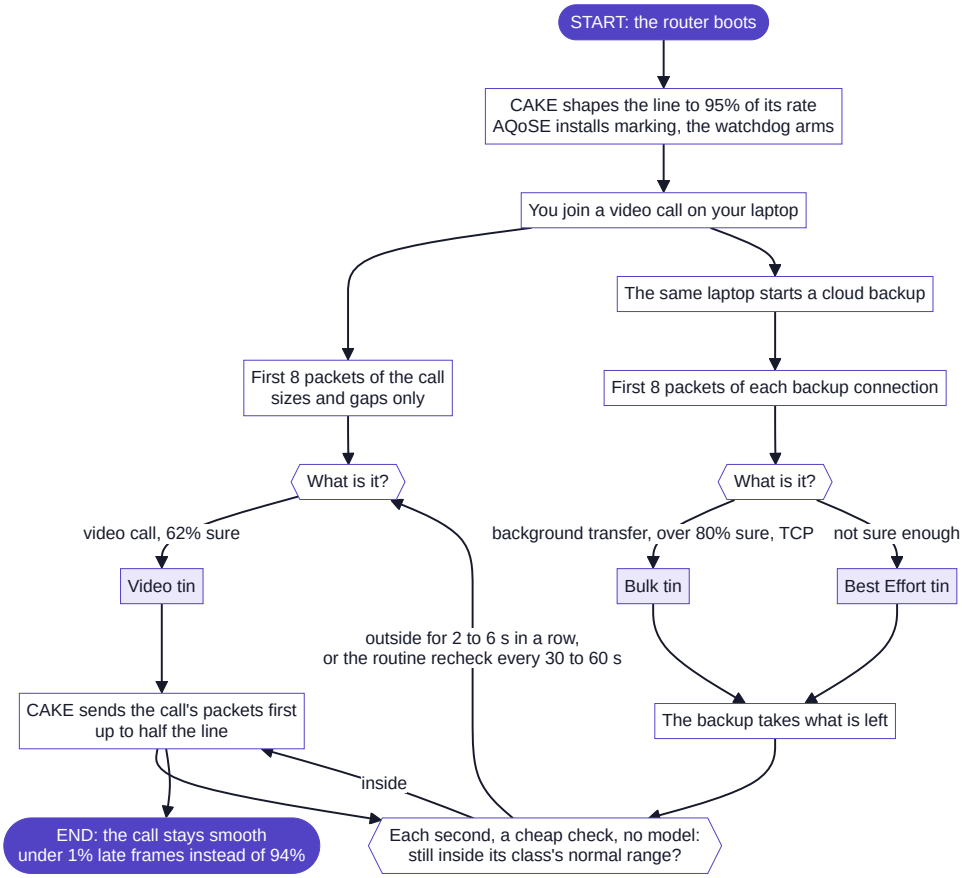


Figure 1: Activity flow for one evening. Tins are CAKE's priority lanes; the numbers come from the replayed WhatsApp call (S12) and the HD-call scenario (S6).

2.2 System architecture

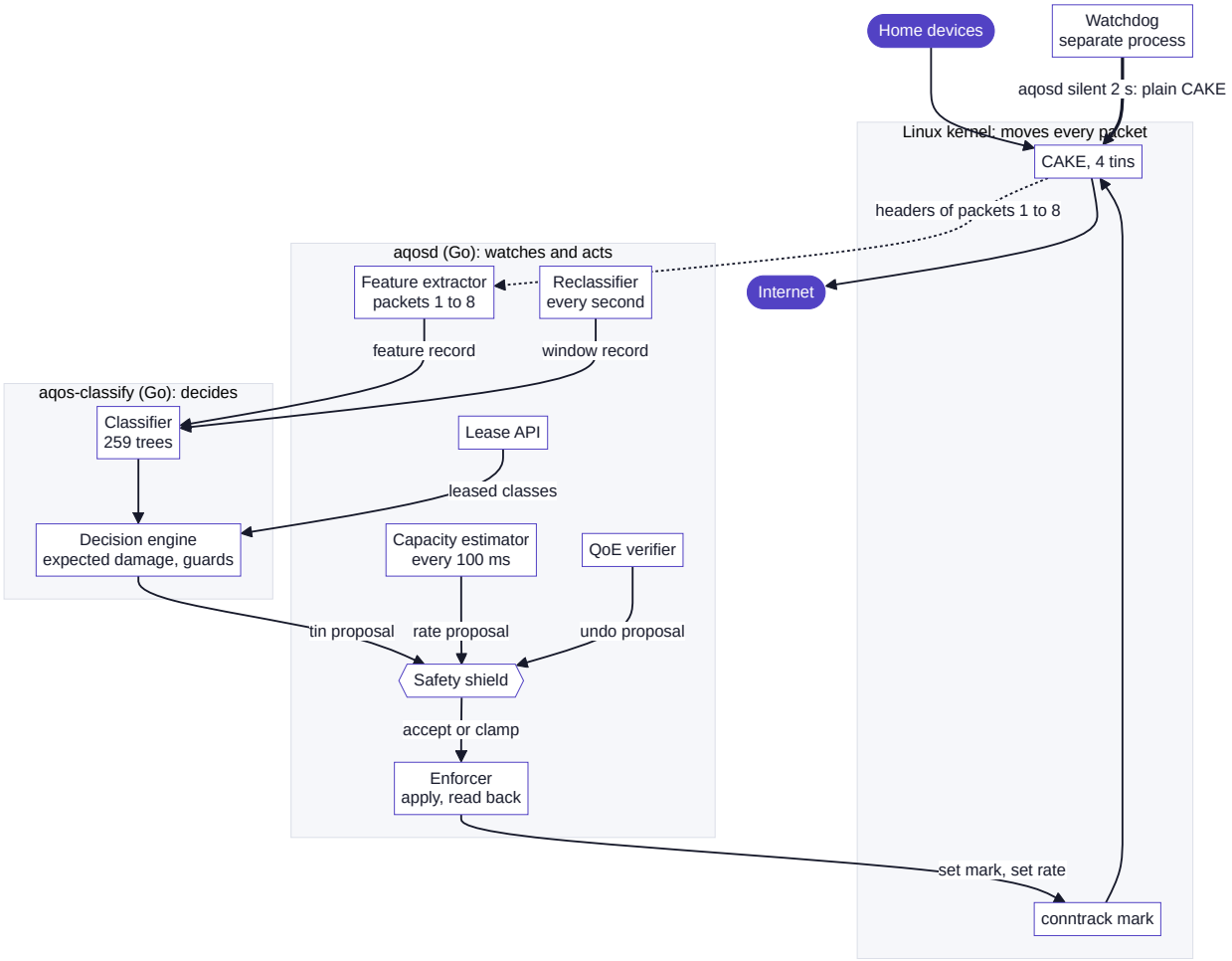


Figure 2: System architecture. Solid arrows carry decisions; dotted arrows carry measurements. Every proposal, from the classifier, the capacity estimator or the verifier, passes the safety shield before the enforcer touches the kernel.

2.3 Layers

Figure 3 places the same parts in six layers, from the devices in a home to the interfaces people use.

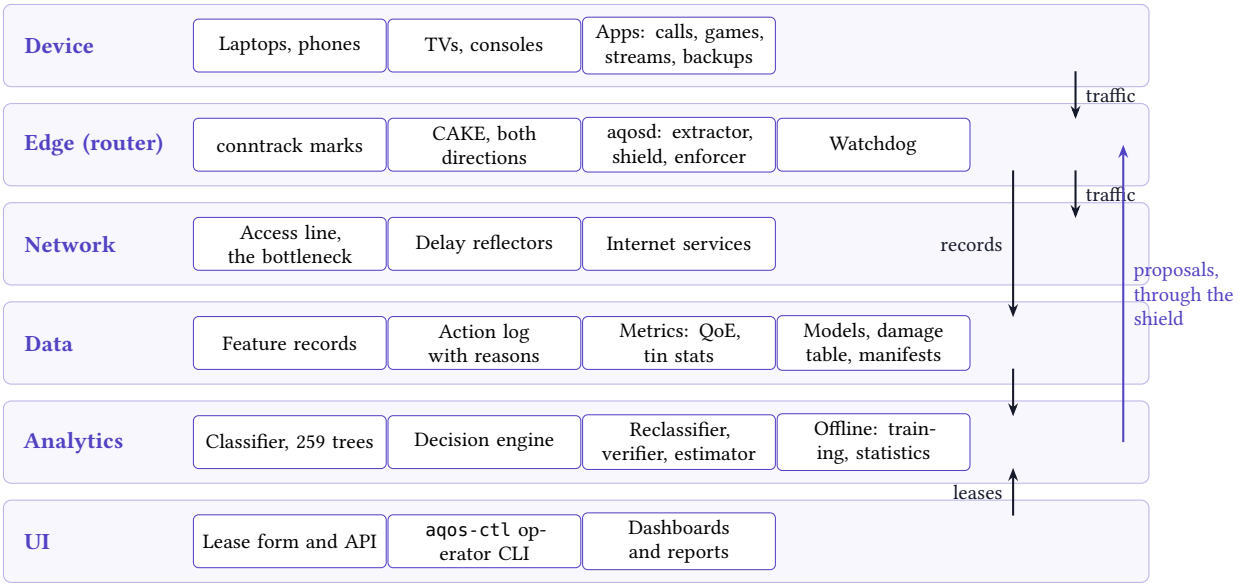


Figure 3: Layered architecture. Traffic flows down from the devices through the router to the line; measurements flow into the data layer and the analytics; decisions flow back up to the router only through the safety shield.

2.4 Principle: no AQoSE code on the per-packet path

The kernel forwards every packet and CAKE queues it. AQoSE writes one label per connection into the kernel's connection tracker (conntrack), and CAKE reads that label for each packet. If AQoSE is slow, wrong or dead, packets still flow; an unlabelled connection is exactly plain CAKE. Figure 4 follows one connection from start to end.

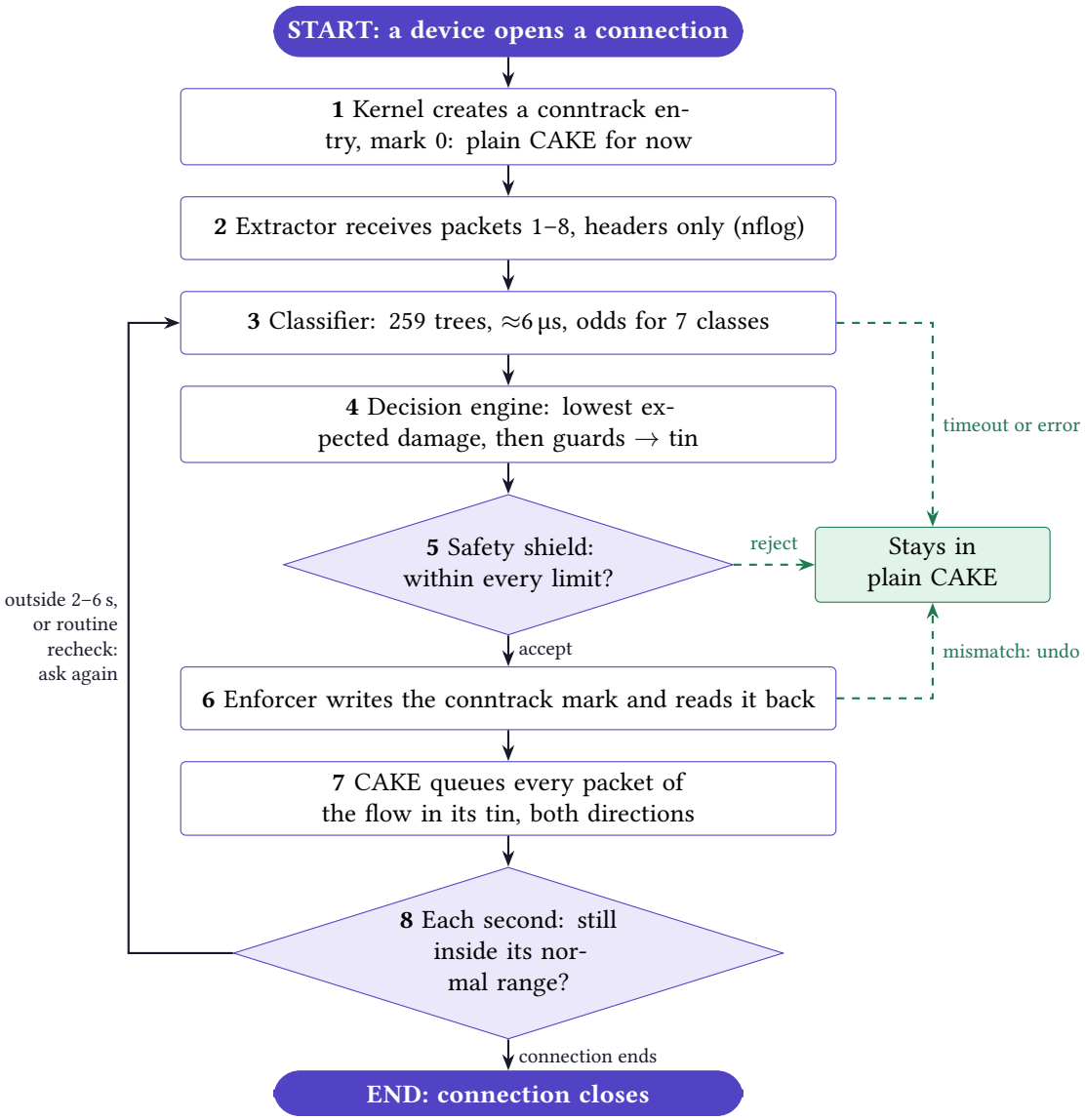


Figure 4: Life of one connection. Solid arrows are the main path; dashed arrows are the ways out, and every one ends in plain CAKE. Alongside run a capacity estimator (every 100 ms), a QoE verifier (after each promotion) and a watchdog (every 250 ms); their proposals pass through the same shield.

2.5 Components

Process	Role
`aqosd` (Go)	The controller: feature extractor, flow table, reclassifier, capacity estimator, verifier, lease API, safety shield, enforcer, heartbeat
`aqos-classify` (Go)	The model and decision rule behind a Unix socket, one JSON object per line
`aqos-watchdog` (Go, standard library only)	Restores plain CAKE when the heartbeat stops
`aqos_ml` (Python)	Training, the threshold rules (B4), traffic generators, experiment runner, statistics
`aqos-testbed` (Go)	Builds the emulated home from network namespaces, rootless

Marking. The label is the low three bits of the conntrack mark; CAKE's `fwmark 0x7` option maps mark n to tin n (1 Bulk, 2 Best Effort, 3 Video, 4 Voice). On upload a postrouting rule copies the label to each packet; on download, packets are redirected to an IFB device whose filter restores it (`ctinfo cpmark`

`0x7`) before a second CAKE instance. Mark 0 falls through to CAKE's own DSCP handling, so "unlabelled equals plain CAKE" holds by construction, and removing two rules turns the whole system off. Verified in the testbed for both directions, IPv4 and IPv6.

Feature extractor. One nftables rule, `ct packets < 9 log group 5 snaplen 64`, sends the first eight packets of every flow to the extractor, headers only; the kernel does the filtering. Packets are timestamped by the kernel. Flow and host identifiers are keyed hashes; addresses never leave the controller. Once a second the controller reads conntrack counters for the nine window features. The same Go package runs offline on captures, so training and live features come from one code path (120 of 120 flows identical).

Model input	Count	Notes
Sizes of packets 1–8	8	IP length; positive LAN to WAN, negative WAN to LAN
Gaps between them	7	microseconds
IP version, protocol	2	
Port class of each end	2	system, registered or dynamic range; raw ports never used
Window statistics	9	packets/s and bytes/s each way, mean size each way, size spread, rate variation, burstiness; missing on a first-packets record

Classifier. LightGBM gradient-boosted trees: 37 boosting rounds for each of 7 classes, 259 trees in all, up to 127 leaves each, learning rate 0.05. Probabilities are calibrated by temperature scaling ($p = \mathrm{softmax}(1.125\,z)$). The trees are exported as data and evaluated in Go; on 2,000 records Go and Python agree to 4×10^{-16} and give the same tin every time. One classification takes about 6 µs in Go, against about 2 ms in the Python service.

Decision engine. For each tin t, expected damage is

$$E[t] \;=\; \sum_{c} P(c)\, D[c][t],$$

where D is a measured damage table (Table 1). The lowest allowed $E[t]$ wins. Rules on top:

- *Voice bar*: Voice only if $P(\text{voice call}) + P(\text{game}) > 0.5$. The table rates Voice as free for most classes, because each was measured alone there; Voice is a quarter of the link.
- *Demotion bar*: Bulk only if the classes whose best tin is Bulk have summed probability over 0.8.
- *Bulk guard*: never demote on a first-packets record, and never a UDP flow. On held-out data sources it cut unseen calls, games and downloads sent to Bulk from 16%, 2% and 10% to 0%, at the cost of leaving UDP peer-to-peer in Best Effort.
- *Ties*: tins within 0.05 are tied; a tie goes to the tin RFC 4594 gives the likeliest class, never to Bulk.
- *Leases*: on a device the user prioritises, the damage of interactive classes in Best Effort or Bulk is multiplied by 5.

Class	Voice	Video	Best Effort	Bulk
Voice call	0	0	0	0
Game, cloud game	0	0	2.5	6
Video call	1.5	0	2	2.5
Streaming	0	1	0.5	1.5
Web and other	0	3.5	0	7
Large download	0.5	4.5	0	7
Background transfer	2	2	1.5	0

Table 1: The measured damage table. Each class was forced into each tin beside a call, a stream, a download and a peer-to-peer transfer, under three conditions (alone on its device, a crowded device, a narrow uplink), two seeds each: 240 runs. A cell is the flow's own loss plus the harm to everything else in the home, rounded to half a point. A download in Video costs 4.5 because it hurts the calls there.

Worked example. A flow rated 45% game and 55% background transfer: Voice $0.45 \cdot 0 + 0.55 \cdot 2 = 1.10$ (barred, since $0.45 \leq 0.5$); Video 1.10; Best Effort $0.45 \cdot 2.5 + 0.55 \cdot 1.5 = 1.95$; Bulk $0.45 \cdot 6 = 2.70$. Expected damage chooses Video. Top label would choose Bulk, and the demotion bar would hold it in Best Effort.

Safety shield. A pure function: proposal and current state in, accept, clamp or reject out. It rejects anything invalid against the JSON contract, stale proposals, a third tin change within 60 s for one flow (the flow is then pinned unlabelled), and more than 50 re-marks per second; it clamps shaper rates to their bounds and to steps of $+10\%$ / -50%, and allows an increase only 10 s after the last. Go's fuzzer generated 10.9 million proposals in 5 minutes with no violation; six bugs planted on purpose were each found in under a second.

Enforcer. Every change is a transaction: snapshot, apply, read back, compare; a mismatch restores the snapshot. Every proposal is logged with its verdict and a plain-English reason, for example "Looks like a video call (62% confident); the Video tin has the lowest expected damage."

Reclassifier. Each class has an envelope: the 5th to 95th percentile of each window feature over its training flows. A flow whose window lies outside on two or more features for two seconds (Voice, Video) or about six seconds (Best Effort, Bulk) is classified again; triggers back off from 5 to 40 s, and every flow is rechecked at a random moment 30–60 s after its last answer.

QoE verifier. Only what a router can see: CAKE's Voice and Video tin delay and the probe delay. Ten seconds before a promotion are compared with two 5 s windows after; if delay rose by more than 5 ms and more than half in both, the change is undone through the shield.

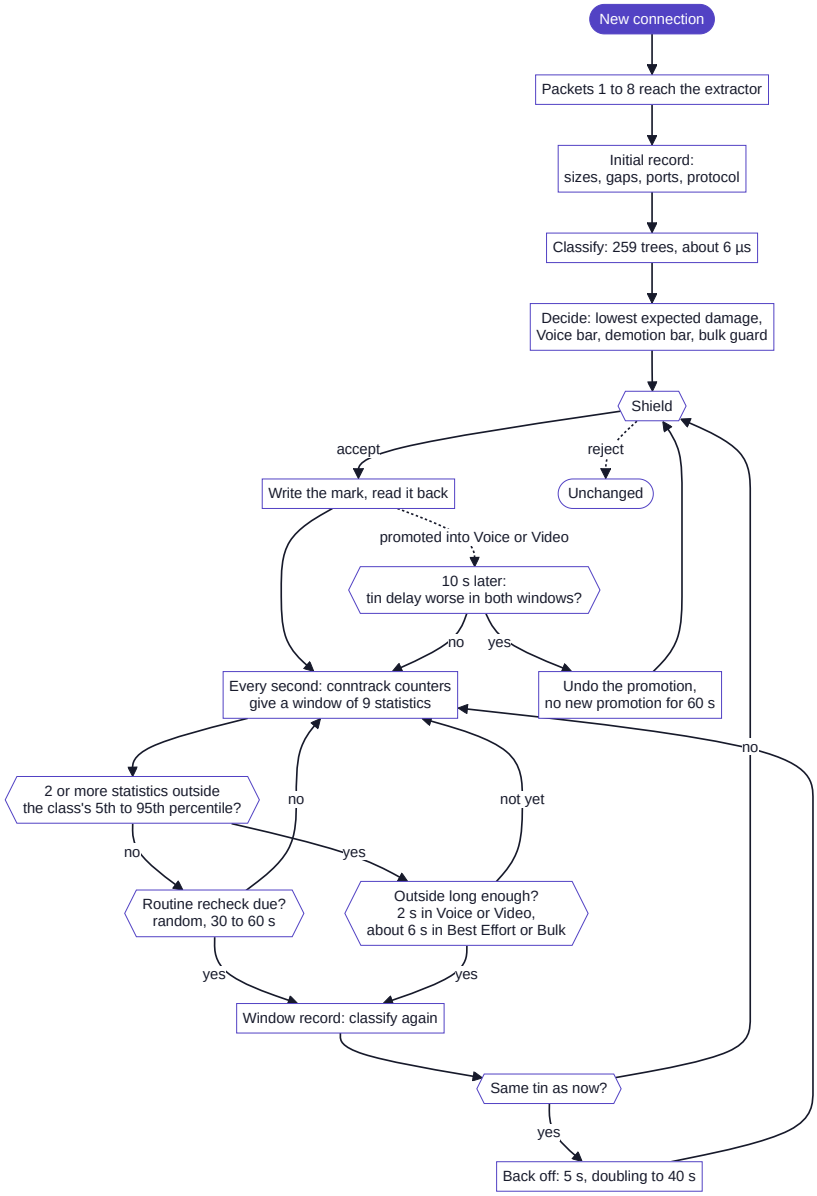


Figure 5: The classify and reclassify loop. A flow is classified once from its first eight packets, then watched every second against its class's normal range. A flow that leaves the range for long enough, or whose routine recheck is due, is classified again from a window record; a promotion into Voice or Video is checked by the verifier ten seconds later.

Capacity estimator. Three UDP reflectors probed every 100 ms; delay above a slowly rising baseline signals a queue beyond the router. The download is cut to 90% of what got through; the upload, whose delivered rate CAKE cannot see, by a fixed quarter. Because round-trip probes cannot tell which direction is queueing, it cuts the download first and touches the upload only if delay persists 2 s later. Raises are one 10% step every 10.5 s at most; with the reflectors silent it changes nothing.

Watchdog. The controller touches a heartbeat file every 250 ms. If the file is older than 2 s, the watchdog removes the marking rules and replaces both CAKE instances with plain CAKE at fixed safe rates.

Intent leases and overrides. A small HTTP API in the controller, with a one-page form, lets a person prioritise a device for up to two hours (at most two leases at once); in the prototype it has no authentication (Section 6). A lease is the user's override of the classifier for that device: it multiplies the damage of leaving

its calls and games in Best Effort or Bulk by five, so an uncertain flow on that device is promoted. An operator can also set a flow's tin directly with `aqos-ctl`. A lease takes effect at the flow's next window record, up to 5 s later.

2.6 Module contracts

Table 2 gives each module's inputs, outputs, state, failure handling and the condition we tested it against. Figure 6 shows the states one connection moves through.

Module	Input	Output	State	On failure	Success, as tested
Feature extractor	Headers of packets 1–8 (nflog); conntrack counters, 1/s	Initial and window feature records	Flow table: 5-tuple, sizes, gaps, hashed ids, label	Buffer overflow: no record, flow stays unlabelled, counted	120 of 120 flows identical live and offline; never more than 8 packets per flow
Classifier	Feature record, 28 numbers	7 calibrated class probabilities	None; model loaded at start	50 ms deadline, queue of 1,024, 200/s cap: flow stays unlabelled	Matches Python to 4×10^{-16}; 6 µs; sealed-test accuracy 0.969
Decision engine	Probabilities, leased classes, record kind and protocol	Proposal: tin, DSCP, reason	None	Invalid probabilities: Best Effort, `fallback`	No call or cloud game sent to Bulk on the sealed test set
Safety shield	Proposal and current state	Accept, clamp or reject, with a code	Per-flow change history, re-mark budget, last rate increase	Panic recovered as reject	10.9 M fuzzed proposals, 0 violations; 6 planted bugs found
Enforcer	Accepted proposal	Mark or rate applied, outcome logged	Snapshot of what it changes	Read-back mismatch: restore snapshot; flow gone: not an error	0 mismatches in 125 frozen-system runs
Reclassifier	Window statistics, 1/s; class envelopes	Recheck requests	Per-flow outside count, back-off	False trigger costs one classification	0.19 tin changes per labelled flow; 2 flows pinned in 80 runs
Capacity estimator	Probe delay, 3 reflectors, 100 ms; CAKE byte counters	Rate proposals	Delay baselines, cut and raise timers	Reflectors silent: hold the rates	S2: under the new line in 2 s, 6% error
QoE verifier	Tin delay and probe delay before and after a change	Regressed, improved or neutral; undo proposal	Pending verdicts	Too few samples: neutral	1 rollback in 80 frozen runs
Watchdog	Heartbeat file age, every 250 ms	Safe state: marking off, plain CAKE at safe rates	Armed or fired	Separate process, standard library only	Plain CAKE 2.03–2.06 s after a kill, in 20 of 20 runs
Lease API	HTTP: device and minutes	Lease with id and expiry	At most 2 leases; stored, survive a restart	Over limit or unknown device: refused with a reason	Applied at the next window, up to 5 s

Table 2: Module contracts.

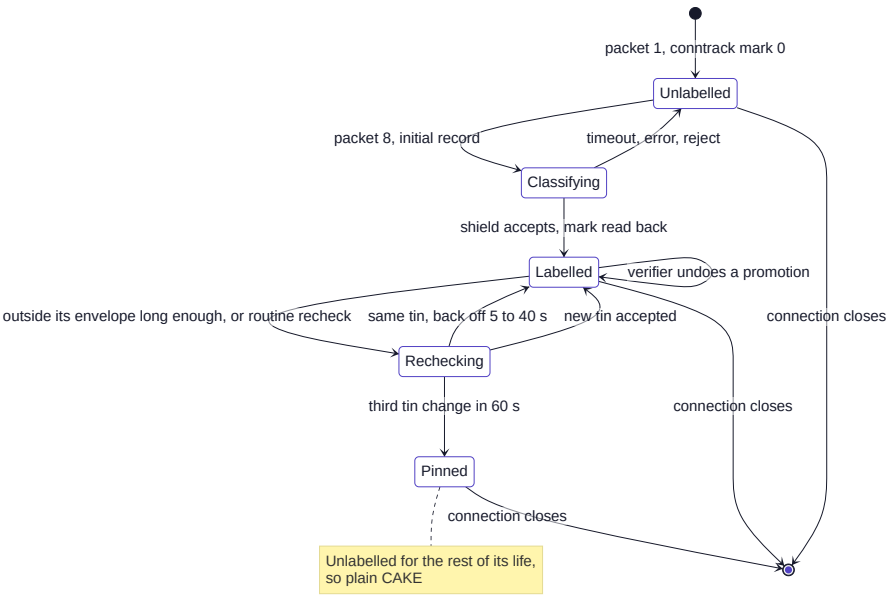


Figure 6: States of one connection. Every failure path ends unlabelled, which is plain CAKE.

2.7 The safety net

Every change, whoever proposes it, passes the same chain of checks (Figure 7). Checks that fail reject the change, and nothing happens; bounds on the shaper rate clamp it instead. After the enforcer applies a change it reads it back and restores the previous state on any mismatch. Two things run beside the chain all the time: the watchdog, and the rule that a flow the classifier cannot answer for stays unlabelled. Both lead to plain CAKE, never to something worse.

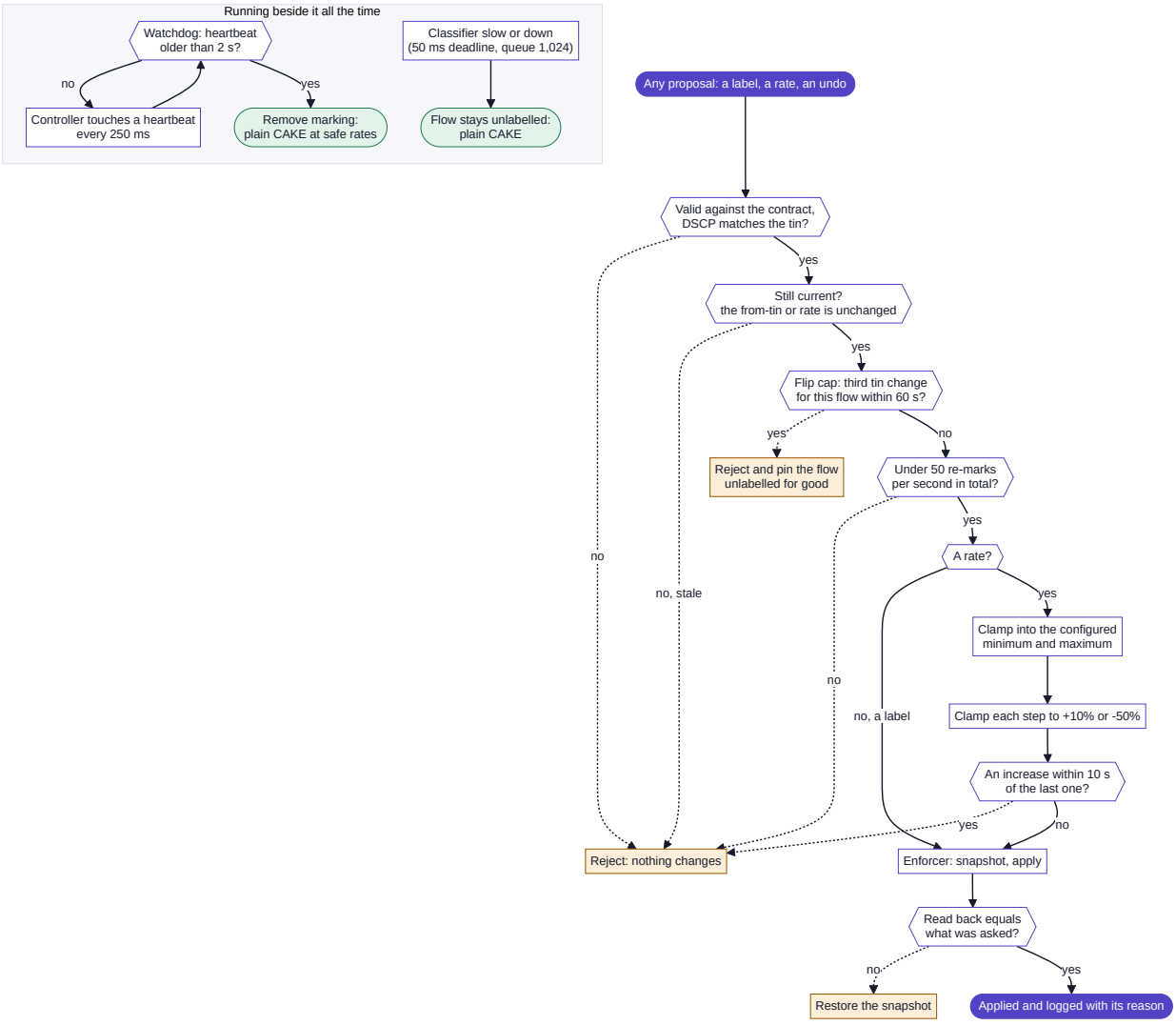


Figure 7: The safety net. Orange boxes are the ways a change is refused or undone; green boxes are the fall-backs to plain CAKE.

2.8 How the line is shared between the four tins

AQoSE never sets a bandwidth itself. It only chooses a tin; CAKE's `diffserv4` mode then shares the shaped rate as in Figure 8 and Table 3. A tin has priority only up to its threshold; traffic beyond the threshold still flows but without priority. Bulk keeps a minimum of one sixteenth of the line so a backup is never starved outright. Inside every tin, CAKE is fair first between devices and then between each device's connections.

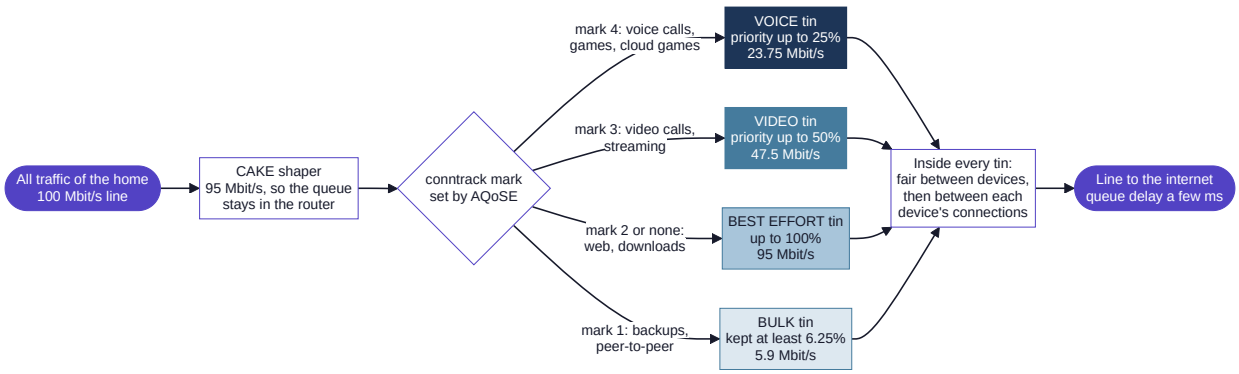


Figure 8: How traffic is distributed on a 100 Mbit/s line shaped to 95 Mbit/s.

Tin	Mark	Share at priority	On a 95 Mbit/s shaper	What AQoSE puts there
Voice	4	up to 25%	23.75 Mbit/s	voice calls, games, cloud games
Video	3	up to 50%	47.5 Mbit/s	video calls, streaming
Best Effort	2 or none	up to 100%	95 Mbit/s	web, downloads, anything unsure
Bulk	1	at least 6.25%	5.9 Mbit/s	backups, peer-to-peer (TCP, over 80% sure)

Table 3: CAKE `diffserv4` thresholds. These shares explain two results: a 24.9 Mbit/s cloud game outgrew Voice's 25% (one fresh seed of S7), and a real call's keyframe bursts outgrew Video's 50% of a 4 Mbit/s upload (S13).

3 Data and model

3.1 Dataset

Version 2 has 246,274 records (first-packets records and five-second windows) from seven sources: ISCX VPN-nonVPN and Tor/non-Tor (2016), the Purdue video-call set (Zoom, Meet, Teams and Webex, 2026), UNSW cloud gaming (2025), Labayen (2020), traffic from the testbed's own generators, and our own captures (JioGamesCloud sessions and six WhatsApp calls on a home line). Records are split by session, never by record: 156,049 for training, 29,250 for calibration, 43,221 for validation, and a sealed test set of 1,213 flows from 183 sessions, opened once, after the system was frozen. Each flow counts once in training however many records it has, and each class counts the same.

Public datasets carry their lab's fingerprint: one sends all streaming through a proxy and all bulk transfers over SSH, so a model trained on it alone would learn the lab. Hence several sources per class, a check of each source before use, and own captures.

3.2 Model choice: LightGBM over a random forest

Both were trained on the same splits and judged by the rule fixed before training: class-balanced expected damage on validation, through the real decision engine. On the v1 validation set they tied (0.141 against 0.146; accuracy 0.944 against 0.942). LightGBM answers in a tenth of the time (1.4 ms against 13.5 ms per record in Python) and is an eighth of the size (1.5 MB against 11.7 MB). On a router those two decide it.

3.3 Offline results

	Validation (v2)	Sealed test set, opened once		
	LightGBM	LightGBM	Tuned rules	qosify defaults
Flows (sessions)	3,609 (372)	1,213 (183)	1,213	1,213
Accuracy	0.972	0.969	0.136	0.119
Macro-F1	0.80	0.73	0.06	0.04
Balanced expected damage, expected-damage rule	0.239	0.270	0.636	1.620
Balanced expected damage, top-label rule	–	0.493		
Video calls sent to Bulk	0%	0% of 221	0%	14%
Cloud-gaming flows sent to Bulk	0%	0% of 47	1%	78%

Table 4: Offline scores. The rules choose among three tins, not seven classes, so their damage is the fairer comparison than their accuracy. Small classes are thin: the test set has 6 voice-call, 8 streaming and 4 large-download flows.

Leave one source out. Trained without one source and tested on it, the model does much worse: damage 1.63 on Purdue video calls and 2.13 on UNSW cloud gaming, against 0.00–0.03 with every source in training. Within its sources the model is excellent; across sources it is not yet. This is the largest risk on the machine-learning side, and the reason for the bulk guard and for the real-call experiment in Section 5.3.

4 How it was tested

4.1 Testbed

One user namespace holds a network namespace per home device, a router, an ISP bottleneck (a token bucket with a 500 ms FIFO, the "bloated modem"), a delay element and the far-end servers. No root privileges are needed. Offloads are off on every link so the router sees real packet sizes. The bottleneck's rate error measured 0.00%. Every experiment in this report ran in this emulation, on one laptop; AQoSE has not been run on a real router or deployed in a real home.

4.2 Traffic and measurement

Eleven generator profiles imitate calls, games, cloud games, adaptive video, browsing, downloads, game updates, backups and peer-to-peer. Every packet carries its sequence number, send time and frame number; a frame is late if its last packet arrives more than 100 ms (calls) or 50 ms (games) after it was sent, or never. A plan is a pure function of scenario, entry and seed, so every configuration sees byte-identical traffic for a given seed. A video call can also replay a recorded trace: the real sizes, timing and first packets of a captured call.

4.3 Scenarios and configurations

S1	A large download starts during a video call, on another device
S2	The line drops from 100 to 20 Mbit/s during a call
S3	Three TVs streaming and a game console on a 25/5 line
S4	A cloud backup fills the upload during a game, on another device
S5	A work call on a laptop that is backing up, busy evening, priority lease
S6	An HD video call on a laptop that is also sharing files
S7	A cloud game on a PC that is also fetching a game update
S8	Traffic that changes: camera off and on, lobby to match, a paused download
S10	The delay probes go silent, then the controller is killed
S11	A family evening on a 50/10 line; written before the freeze, never run during development
S12, S13	Real WhatsApp video calls, replayed, on a laptop sharing files (50/4 line); written after the freeze
S14	S6 entirely over IPv6; written after the freeze

Configurations: **B0** no queue management; **B2** plain CAKE at 95% of the line; **B3** CAKE with cake-autorate 3.2.2, unmodified, default delay thresholds, our estimator's rate range and reflectors (`config.b3.sh` in each run); **B4** CAKE with qosify-style threshold rules; **O1** AQoSE choosing the tin of the likeliest class; **O2** AQoSE in full.

4.4 Protocol

Each run is 20 s of warm-up and 180 s measured. Comparisons are paired by seed. We report medians with 95% bootstrap intervals, Wilcoxon signed-rank tests with rank-biserial effect sizes and Holm correction for comparisons not named in advance. Ninety A/A runs (the same configuration and seed twice) measured noise: identical summed cost in eight of nine scenarios, up to 0.14 in S8. A difference inside that band counts as a tie.

The freeze. During development, scenarios exposed problems that were fixed; the system was therefore partly tuned on its own tests. On October 15 at 07:41 the system was frozen (code, model, damage table and every setting). All results in Section 5 come from the frozen system: seeds 0–9 rerun; fresh seeds 10–14 and S11 run once; S12, S13 and S14 run once. All 492 runs after the freeze were valid.

5 Results

5.1 The frozen system, seeds 0–9

		No QoS	CAKE	Rules	Top label	**AQoSE**	autorate
S2	call late %	77.0	55.5	55.6	0.9	**0.9**	4.6
	call p95 latency, ms	507	502	499	18.7	**19.5**	95.9
S5	call late %	94.1	93.3	78.7	0.4	**0.1**	93.3
S6	call late %	94.3	94.3	94.4	0.6	**0.6**	
	call p95 latency, ms	504	1,786	2,063	18.2	**18.2**	
S7	game late %	83.7	83.2	19.4	0.6	**1.0**	
S8	call late %	65.3	61.8	30.8	0.0	**0.0**	
S1, S3, S4, S10	late %	53–78	0.0	0.0–0.3	0.0	**0.0**	

Table 5: Medians over ten seeds. cake-autorate was run on S2 and S5 only.

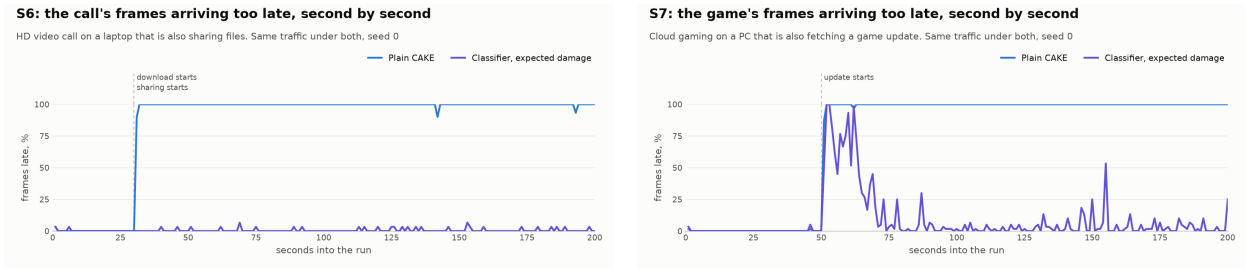


Figure 9: One run each of S6 (left) and S7 (right), second by second. When bulk traffic starts on the same device, plain CAKE lets almost every frame arrive late; AQoSE had already placed the call or game in its tin.

Summed over the nine scenarios, the number of flows left unusable is 12.21 with no queue management, 5.54 with plain CAKE, 6.61 with threshold rules, 0.70 with top label and **0.69 with AQoSE** (medians of ten seeds). The ratio to CAKE depends on the scenario mix and is a summary, not a claim about homes in general. Seed by seed, AQoSE was better than plain CAKE in 49 runs, tied in 41 and worse in none; better than the threshold rules in 52 and tied in 38.

Why plain CAKE's latency exceeds no queue management in S6. The 1,786 ms is the call's upload (its download: 860 ms). Without queue management the modem's 500 ms buffer caps every packet's delay. Under plain CAKE the laptop's 19 Mbit/s upload is shared between the call and ten file-sharing connections, so the call gets about 1.7 Mbit/s of the 5 Mbit/s it sends; most of the excess is dropped (65% of its upload packets in seed 0) and the rest waits in the call's own queue for close to two seconds. The baseline works as designed; the share is the problem.

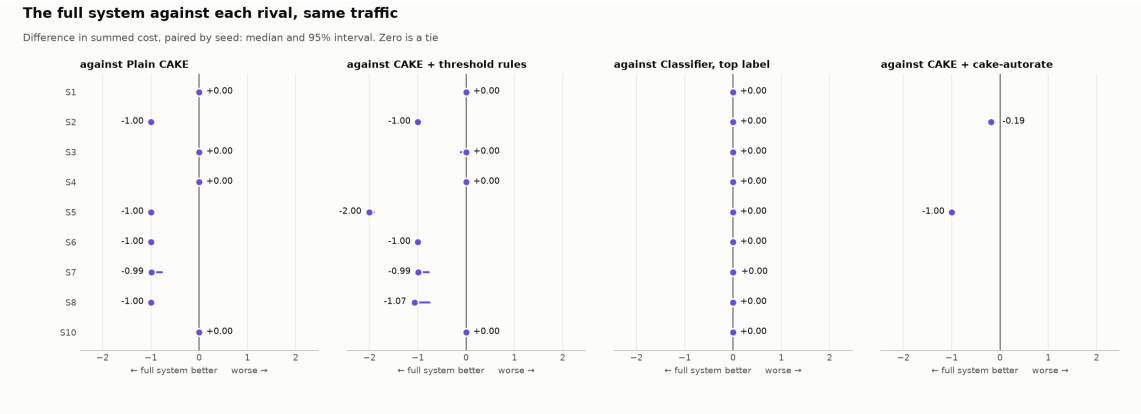


Figure 10: AQoSE against each rival, paired by seed: median difference in summed cost and 95% interval. Left of zero AQoSE is better.

Against cake-autorate. On S2, where the line slows, cake-autorate cuts late frames from 55.5% to 4.6%; AQoSE's built-in estimator reaches 0.9% with a p95 latency of 19.5 ms against 95.9 ms. On S5, where the problem is a call sharing its device with a backup, cake-autorate does nothing (93.3%), because no rate setting fixes it. AQoSE was better in all 20 paired runs.

Where it ties. In S1, S3, S4 and S10 every interactive flow is alone on its device; CAKE's fairness already protects it, and AQoSE matches CAKE. The damage measurements predicted this before the scenarios ran.

Fail-safe. In all 20 S10 runs, the watchdog restored plain CAKE 2.03 to 2.06 s after the controller was killed.

5.2 Fresh seeds and an unseen scenario

On seeds 10–14, run once, AQoSE was better than plain CAKE in 21 paired runs, tied in 29 and worse in none, and never worse than the threshold rules. The wins held: S6 94.3% → 1.0%, S2 55.5% → 0.9%, S5 78.1% → 0.0%, S7 83.1% → 0.4% late frames (medians). Unusable flows over the ten scenarios: plain CAKE 4.99, threshold rules 7.86, AQoSE 0.99. With five seeds no single comparison reaches significance on its own (Wilcoxon's smallest possible p is 0.0625); the direction agrees on every scenario.

S11. On the scenario written before the freeze and never run during development, plain CAKE already protects everything (0.7% late frames) and AQoSE ties it (0.0%); the threshold rules ruin the call (69%). S11 shows AQoSE stays safe on a home it was not tuned for; it is not a new win over CAKE.

One failure. In one fresh seed of S7 the generator drew a 24.9 Mbit/s cloud game. AQoSE placed it in Voice, as the taxonomy says, but `diffserv4` gives the Voice tin priority only up to a quarter of the shaped rate (23.75 Mbit/s), so the game lost priority and 72% of its frames were late, no better than plain CAKE. A high-bitrate interactive flow belongs in Video, which has half the rate; this is a design change for the next version.

5.3 Real calls, replayed

Because the testbed's generators are in the model's training data, the live wins might show the classifier recognising its own lab traffic. Two real WhatsApp video calls captured on a home line were therefore replayed packet for packet on an emulated laptop sharing files. Replay here means the testbed's generator

re-sends the recorded packet sizes and timing inside the emulation; the WhatsApp app was not running, and the replayed call does not adapt its bitrate as the real app would. This is real traffic, not a live deployment.

Late frames (median of 5 seeds)	CAKE	Rules	Top label	**AQoSE**	AQoSE, v1 model
S12: call the frozen model never trained on	57.0%	42.7%	2.3%	**2.4%**	6.6%
S13: larger call, in the model's training data	78.7%	97.3%	14.2%	**14.4%**	30.9%

Table 6: The last column uses the model from before the freeze, which never saw WhatsApp at all.

The frozen model labelled both calls "video call" (61–66% confident) within 0.13 s in all 20 runs and kept them in Video throughout. The model that never saw WhatsApp labelled them "voice call" (91%), which still protects them, but the reclassifier then moved the calls between Voice and Video several times, which cost quality (6.6% and 30.9% late). S13's remaining late frames are almost all on the 4 Mbit/s upload: the call's real keyframe bursts exceed the half of the link the Video tin may use at priority. The tin was right; the limit is CAKE's share.

5.4 Is expected damage the reason?

The classifier, not the decision rule, does most of the work: top label alone takes unusable flows from 5.54 to 0.70, and expected damage to 0.69. To test the decision rule directly, the classifier was weakened by multiplying every input by $\exp(\mathcal{N}(0, \sigma))$.

Noise σ	Accuracy	Offline damage		S6 call late %, live	
		top label	expected	top label	expected
0	0.976	0.302	0.235	0.7	0.5
0.5	0.679	0.691	0.577	9.6	54.4
1.0	0.610	0.858	0.789	62.1	94.3
1.5	0.551	0.915	0.863	94.1	94.3

Table 7: Offline, expected damage beats top label at every noise level. Live (three seeds per cell) it did not: with a weaker classifier it left the S6 call worse.

Why it lost at $\sigma = 0.5$. Not by misplacing the call: in all six runs both rules put the call in Video at once and kept it there for at least 178 of the 200 s. The late frames came from file-sharing and download connections that the noisy classifier misread and promoted into Voice, which has priority over Video, and into Video beside the call. Both rules did this, and the runs vary widely: 7.6, 73.1 and 9.6% late by seed with top label, 13.3, 54.4 and 67.0% with expected damage. The noise is not paired between the two rules, so with three seeds the medians mostly show which runs drew more misread connections. We read this as no live evidence for expected damage, not as evidence that it hurts.

We therefore do not claim a live benefit for expected damage. It is a principled way to combine uncertain predictions with measured costs, and offline it lowers damage by 22% with the full classifier and by 6–17% with a weakened one, but in the live tests its value is as the frame for the guards, not as the source of the wins.

5.5 Bulk progress and fairness

Protecting a call must not starve everything else. Table 8 compares every bulk transfer's throughput under AQoSE with the same transfer under plain CAKE on the same seed. No transfer was starved more than under plain CAKE (a second under 100 kbit/s; the only scenario with many such seconds is S8, whose

update is paused on purpose, identically in both). Jain's index over the devices' throughput is unchanged within ±0.01 in every scenario, on both the main and the fresh seeds.

	Bulk transfer	CAKE, Mbit/s	Rules	AQoSE	AQoSE / CAKE
S1	download	88.5	88.5	88.5	100%
S2	download	37.8	37.9	36.9	97%
S4	backup, upload	8.9	8.9	8.9	100%
S5	backup, upload	2.8	3.3	2.3	83% (lowest 54%)
S6	download	43.8	42.4	41.7	95%
S6	file sharing, down	40.7	40.3	42.4	104%
S6	file sharing, up	14.6	14.5	10.9	75%
S7	game update	66.2	66.8	65.0	98%
S8	sharing and update	31–53	31–55	31–53	98–101%
S10	download	88.5	88.5	79.2	90%
S11 (fresh)	backup, download, update	6–20		6–21	96–105%

Table 8: Bulk throughput, medians of ten seeds (S11: five fresh seeds). The bulk traffic that gives way is the traffic sharing a device with a call: the backup in S5 and the file sharing's upload in S6. In S10 the download loses 10% for a different reason, given in Section 8.

5.6 Jitter, loss and throughput

		Jitter, ms			Packet loss, %		
	Flow	CAKE	Rules	**AQoSE**	CAKE	Rules	**AQoSE**
S2	call, down	0.5	0.5	0.9	0.29	0.32	**0.00**
S5	call, up	8.0	15.3	**3.0**	24.0	53.6	**0.01**
S6	call, down	27.3	3.2	**0.2**	91.1	52.8	**0.00**
S6	call, up	6.6	8.7	**0.6**	65.4	69.7	**0.03**
S7	cloud game, down	3.1	0.1	0.3	78.9	16.7	**0.01**
S8	call, up	6.6	8.7	**0.5**	12.2	6.2	**0.00**

Table 9: Medians of ten seeds; every other call and game flow is under 1 ms jitter and 0% loss with all three. The calls' throughput follows: in S6 the call received 432 kbit/s of its 5 Mbit/s under plain CAKE and 5,051 kbit/s under AQoSE.

5.7 Link estimation

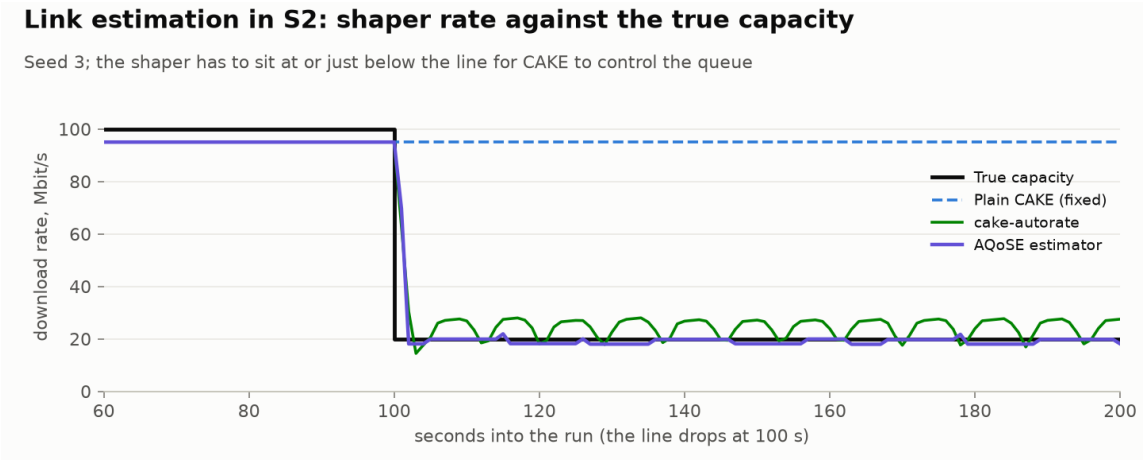


Figure 11: S2, seed 3: the download shaper against the true line rate. Plain CAKE stays at 95 Mbit/s, nearly five times the line after the drop.

	Detects the drop	Under the new line	Steady-state error	Time above the line
AQoSE estimator	within 1 s (360 ms in its log)	2 s	**6%**	**2%**
cake-autorate	within 1 s	3 s	20%	76%

Table 10: Medians of ten S2 runs, from per-second samples. Error is the mean absolute difference from the true rate from 10 s after the drop. cake-autorate ran with its default delay thresholds, base and maximum rate at 95% of the line, minimum at a tenth, and the testbed's three reflectors pinged with `ping`, the same range and reflectors as our estimator; tuning those thresholds could narrow its error, which we did not try.

5.8 Policy stability and adaptation speed

Over the 125 frozen-system runs the policy stayed calm. A connection was labelled within about 0.13 s of starting, after its eighth packet. Labelled flows changed tin 0.19 times each on average, about once a minute across a whole home; the flip cap pinned 2 flows in 80 runs; the verifier undid 1 change; and the classifier service never timed out, overflowed its queue or disagreed with a read-back. The capacity estimator changed the rate 5 times per run on average.

5.9 IPv6 end to end

Marking had been verified on IPv6 in unit tests, but every scenario ran on IPv4. S14 is S6 with every connection over IPv6: the call, the file sharing and the download. It was written and run after the freeze, three seeds each with plain CAKE and AQoSE, all valid. The classifier recognised the call as a video call (97% confident) at 0.01 s, although IP version is one of its inputs and only 1% of its training records were IPv6. This is not the model reacting to IP version: the same call over IPv4 in S6 was also labelled at 97% in 9 of 10 seeds (82% in the other), and setting only the IP version of all 43,221 validation records from 4 to 6 changed the top label of one record and the probabilities by 10^{-5} on average. (The 61–66% in Section 5.3 is a different call, a real WhatsApp recording.) The call's late frames fell from 94.3% to 0.3%, its p95 latency from 1,788 ms to 19 ms and its download packet loss from 90.9% to 0%. The bulk transfers kept 73–105% of their plain-CAKE throughput, and Jain's index between the devices was 0.999 with plain CAKE and 0.996 with AQoSE.

5.10 Dashboard

Every run records the metrics a dashboard needs: per-packet latency, jitter and loss for each call and game, per-second throughput for every flow, and CAKE's delay, backlog and drops for every tin. Figure 12 is the run dashboard generated from one run of S5 by `aqos_ml.evidence` (command in the appendix). The repository also holds a Grafana dashboard and a Prometheus exporter that tails a running experiment; they were generated and checked against the metric contract but not used for the results here.

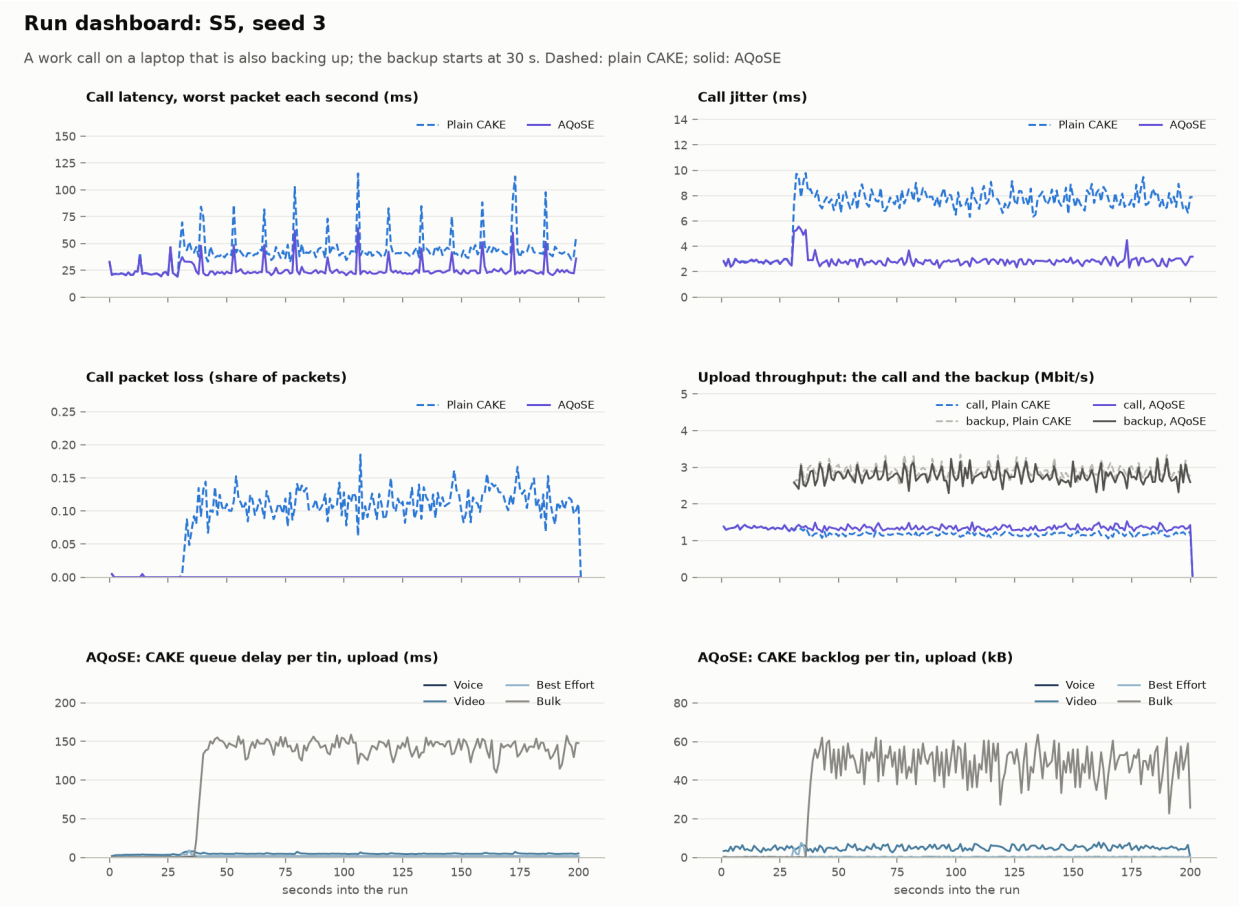


Figure 12: Run dashboard, S5 seed 3. When the backup starts at 30 s, plain CAKE's call loses 10–15% of its packets and its jitter triples; with AQoSE the call is untouched, and the backup's queue sits in the Bulk tin.

5.11 Cost

Component	CPU, % of one core (mean over scenarios)
Controller `aqosd`	3.6 (3.2–4.4)
Classifier service (Go)	0.04
Delay reflector	0.22
Watchdog	0.06
cake-autorate, for comparison	1.2 (0.8–1.6)

Table 11: Historical Go-runtime measurements on the reference laptop, averaged over the original scenarios. These are not Rust measurements; router CPU use remains unmeasured.

5.12 Rust runtime efficiency and stability update

The production runtime now defaults to Rust: flow extraction, model inference, the controller, shield and enforcer, capacity probes, reclassification, verification and rollback, intent API, metrics tools and the independent watchdog. The trained model is unchanged. The namespace testbed remains Go, research tooling remains Python, and the previous Go runtime is retained for comparisons.

Measurement	Go	Rust	Change
Model inference (μs)	9.34	6.43	31% less time
Features, inference and decision (μs)	11.02	6.50	41% less time
Classifier request latency, median (μs)	39.98	19.26	52% lower
Classifier resident memory (MiB)	11.57	5.45	53% lower
Controller CPU per short run (CPU-seconds)	0.695	0.395	43% lower
Offline extraction of 50,000 flows (ms)	326	244	1.34× faster

Table 12: Separate Go–Rust host benchmarks. Values are medians; improvements do not combine into an overall network speedup. The extraction measurement used an earlier Rust migration build.

Method and scope. Classifier tests used the same model and 2,000-record corpus over seven repetitions, with each service pinned to one CPU. Request latency was measured on a warmed persistent local socket; memory is a post-request resident-set snapshot, not a peak. Extraction used seven alternating trials on 400,000 packets, with identical feature output verified before timing; it measured initial-flow features only. Controller CPU came from three paired short O2 runs with the Rust classifier held fixed and excludes child `tc`/`nft` processes. The first Go trial overlapped the end of a regression suite; the reported Go median came from a later trial. These compare implementations with different data layouts and allocation choices, not programming language alone.

Six-hour stability. Scenario S903, O2, seed 23 ran for 21,600 measured seconds plus warm-up, with 30 Hz gaming traffic, a two-connection bulk download, five new short flows per second and repeated link-capacity changes. It passed with 53,251 label applications and 2,039 shaper-rate changes. There were no enforcement errors, read-back mismatches, classifier timeouts, capture overflows, service replacements or watchdog recoveries. Controller RSS peaked at 6.12 MiB; the median of its final sampling window was 4.93 MiB. The run exited successfully and removed the testbed. An earlier attempt stopped after about 31 minutes at the disk-space guard; that attempt is not counted as a six-hour pass. The exporter was not exercised in the soak.

What this changes in the report. These results establish lower implementation overhead in the measured workloads and successful operation for six hours. They do not replace the original 3.6% scenario-average controller CPU figure, the 0.13 s connection-labelling time, or any call, game, throughput or accuracy result. In particular, the report's existing *about six microseconds* inference claim is not improved by comparing it with 6.43 μs; the measured speedup uses the matched 9.34 μs Go baseline. The original scenarios and seeds must be rerun with Rust before those tables can be updated. Saturation performance, OpenWrt cross-compilation, physical-router measurements and proof of leak freedom remain outside this evidence.

Evidence. Raw trials, methods and binary hashes are recorded in the local repository artifacts below. The rebuilt classifier and controller binaries match their benchmark hashes; the final extractor still needs a fresh benchmark.

```
reports/rust-optimization.json
reports/rust-runtime-controller-comparison/comparison.json
reports/rust-runtime-benchmark.json
reports/rust-six-hour-validation.json
reports/rust-resume-build-verification.json
```

6 From prototype to product

Security and privacy. AQoSE reads packet headers only, never payload, and never decrypts. Flow and device identifiers are keyed hashes; addresses never leave the controller, and the action log holds only hashed ids. Nothing in the system needs a credential, and the repository holds none (its full history was scanned before it was made public; API tokens and cookies are excluded by `.gitignore`). The lease API has no authentication or access control in the prototype. It is off unless the controller is started with `-intent <address>`; in the testbed it listens only on the router's loopback address, 127.0.0.1. The code does not itself restrict the address to loopback, so in this form the API must not be reachable from a home network or any network beyond the controlled test environment. Before any such deployment it needs authentication and access control: the router's own administrator login over HTTPS, rate limits on the API, and a signed model file: today the model's version and the hashes of its dataset and damage table are recorded, but the file is not signed. In the testbed everything runs without root in a user namespace; on a router the controller needs network-administration capability for tc, nftables and conntrack, and nothing more.

Performance. The historical Go study measured 3.6% of one laptop core for the controller and 0.04% for the classifier. The Rust benchmarks in Section 5.12 establish lower component overhead, but do not provide replacement scenario averages or router CPU figures. Router measurements and saturation tests remain necessary to locate the limiting component at high line rates.

Hardware. Deployment requires CAKE, nftables, the `ctinfo` action and IFB devices. The production programs are now Rust, and the model is a 2 MB file of trees. Selecting an OpenWrt target, cross-compiling with its toolchain and validating the required kernel facilities on the device remain outstanding. Hardware or software flow offloading must be off, since offloaded packets bypass the queue that CAKE manages. ISP-supplied routers with closed firmware would need the vendor's integration.

Operations. Models are versioned directories, and every run records the model, damage table and code version it used, so a model update can be rolled out to a few routers first, compared on tin delay and rollback counts, and rolled back. The watchdog is the kill switch. Telemetry, if any, should be opt-in and aggregate (counts of decisions and tin delays, no addresses). Thresholds, bounds and the damage table are configuration files, not code.

Not assessed. Certification of a router product, regulatory review of traffic management in an ISP-deployed version, and integration with proprietary router firmware or hardware offload engines.

7 What we learned

- **A tin only matters when the flow has company on its device.** The first damage measurements had each flow alone, and every tin looked alike. That reshaped the scenarios and the claim.
- **A wrong label is not a wrong tin.** Calling a video call a voice call harms nothing; both are protected. We measure placements, not only labels.
- **The damage table was wrong three times**, each time exposed by a scenario: no reward for demoting a backup beside a call; a call apparently better off in Bulk; Voice rated free for everything.
- **Simulations share their author's blind spots.** The capacity estimator passed every simulated test and failed four times on the real testbed; each failure is now a regression test.
- **With nothing to go on, do nothing.** Lowering the rate when the probes went silent broke a call on a healthy line; the estimator now holds still.
- **Public data is not automatically good data**: each source carries its lab's fingerprint.

8 Limitations

- **Emulation only.** One laptop, emulated links from 4 to 100 Mbit/s; no real router, no real home, no Wi-Fi.
- **Generated and replayed calls do not adapt.** A real app lowers its bitrate when squeezed; ours lose frames. CAKE's failures are therefore larger here than they would be for real apps.
- **Weak across unseen sources.** Leave-one-source-out damage is high; the bulk guard keeps such traffic out of Bulk, but a call can still be left in Best Effort.
- **Partly tuned on its own tests.** Seeds 0–9 include those the fixes were found on; the fresh seeds, S11 and the real-call scenarios are the answer, and they are few.
- **Thin classes.** 6 voice-call, 8 streaming and 4 large-download flows in the sealed test set.
- **The cost scale is self-graded**: the same scale the damage table is built from. The raw measurements (late frames, latency) are the evidence.
- **Voice tin share.** A cloud game above a quarter of the line loses priority in Voice (one fresh seed of S7).
- **VPN traffic is out of scope.** A VPN carries all of a device's connections inside one encrypted tunnel, so no classifier on the router, AQoSE included, can tell a call from a backup inside it. We did not test VPN traffic. Tunnels over UDP, which most modern VPNs use, are never demoted and so get plain CAKE; tunnels over TCP must be excluded from demotion before any deployment.
- **Probe targets returning.** In S10, when the delay reflectors came back after an outage, the estimator read their return as a queue and halved the download rate for about 30 s; the download lost 10% of its throughput, the call nothing. A fix is needed.
- **Not measured:** saturation under a flood of short flows, gigabit rates, router CPU and memory. The Rust soak exercised five new short flows per second for six hours; it did not establish maximum capacity.
- **Leases** take effect at the flow's next window, up to 5 s, not the 1 s target.

9 Next steps: from emulation to real homes

Everything so far ran in emulation. These are the steps that would turn the prototype into something installed and tested in homes, each with how it would be tested and what would count as success.

1. **Run it on a real router.** Cross-compile `aqosd`, `aqos-classify` and the watchdog for an OpenWrt router (ARM), which already ships CAKE and nftables. *Test*: the same scenarios through the router at 25, 100 and 300 Mbit/s, measuring CPU, memory, classification latency and throughput. *Success*: under 10% of one router core at 100 Mbit/s, no loss of throughput against plain CAKE, and the same wins as in emulation.

2. **Test with real applications, live.** Real Zoom, Meet, WhatsApp and cloud-gaming sessions on laptops and phones over Wi-Fi, next to real backups and downloads, instead of generated or replayed traffic. *Test*: read the apps' own quality statistics (WebRTC `getStats`: frame rate, freezes, bitrate), so the result includes the apps lowering their quality. *Success*: fewer freezes and a higher sustained bitrate than plain CAKE, on the same evening's traffic.

3. **A field trial in homes.** Five to ten volunteer homes for four weeks, switching between AQoSE and plain CAKE on alternate days so each home is its own control. *Measure*: router-side tin delay and probe delay, an opt-in note from users after calls ("did the call stutter?"), and the router's action log. *Success*: fewer reported bad calls on AQoSE days, and no home worse off.

4. **Fix the two known gaps.** Send interactive flows above a set share of the line to Video instead of Voice, and calm the reclassifier for flows the model is unsure about. *Test*: rerun S7 with 20 to 30 Mbit/s cloud games and the v1 real-call runs; then repeat the whole frozen protocol.

5. **Broaden the data.** Captures from the team's and volunteers' homes across Indian ISPs and current apps, at least five sources for every class. *Test*: leave-one-source-out damage below 0.5 for every source, and a new sealed test set opened once.

6. **Stress and coverage.** A flood of short flows, many devices at once, IPv6 on every scenario, and gigabit lines. *Success*: never worse than plain CAKE in any of them.

7. **Make it usable.** A page in the router's web interface (LuCI) to prioritise a device for an hour, and a readable log of why each connection was placed where it was.

10 Conclusion

Home routers already have a very good queue manager in CAKE, and AQoSE does not try to replace it. It answers the one question CAKE cannot: which of the connections on a busy device is the call or the game. A small, calibrated tree model that reads only the headers of a connection's first eight packets is enough to answer it, in about six microseconds, and the answer is written once per connection as a label the kernel already knows how to use. No AQoSE code sits on the per-packet path.

The evidence, all of it in emulation, is consistent. Where CAKE's fairness cannot help, a call or a game sharing its device with bulk traffic, late frames fell from 83–94% to around 1% across the main scenarios, on ten seeds and again on five fresh seeds the system had never seen. Recordings of real WhatsApp video calls, replayed packet for packet inside the emulator, were recognised within a fraction of a second and their late frames fell from 57% to 2.4% and from 79% to 14%. Where CAKE already works, AQoSE ties it, including on a home it was never tuned for. In 140 paired runs it was never worse than plain CAKE, never worse than the threshold rules existing tools use, and better than cake-autorate in all 20 runs where both were tested, 10 of them where the line's speed changed. On the sealed test set of real traffic (1,213 flows from 183 unseen sessions) it labelled 96.9% of flows correctly, but its macro-F1 is 0.73 and some classes have only 4 to 8 test flows, so accuracy alone overstates how well it generalises; it sent no call or cloud game to the slow lane, where qosify's defaults sent 78% of cloud-gaming flows.

The work also showed what does not carry the result. The expected-damage rule is a sound way to combine uncertain predictions with measured costs, but in live tests the classifier did the work: recognising the traffic at all is what fixes CAKE's blind spot. The model is much weaker on data sources it has never seen, CAKE's tin shares limit what any label can do for very large flows, and every result so far comes from one laptop.

The system is built to fail safe: every change passes a fuzz-tested shield, is read back, can be undone by a verifier that sees only what a router sees, and a separate watchdog returns the router to plain CAKE within about two seconds. That makes the next step low-risk: a real router in real homes, where the question is no longer whether the idea works in emulation but how much it helps people on their own calls.

A Reproducing the results

The code, scenarios, the two replay traces, the trained models and the reports are public at `github.com/shreyashsri79/aqose`. Raw run directories are not in the repository because of their size; the commands below regenerate them. The datasets are not in it either. The public sources are downloaded by `scripts/fetch-data.sh` (some sites serve only a registered browser session), but our own captures (three JioGamesCloud sessions and six WhatsApp calls) are not published, so dataset v2 and the sealed test set cannot be rebuilt exactly from the repository alone; the trained model built from them is in it. Commands run from the repository root; the Python package is not installed, so Python modules need `PYTHONPATH=ml`.

```
make venv && make build && make check   # Rust runtime, Go testbed, tests
make testbed-check                      # tests against the real testbed
make build-go-reference                 # historical Go runtime in bin/go
make fuzz-go-reference FUZZTIME=300s     # historical shield fuzzer
export AQOS_RUNTIME=go AQOS_SERVICE=go   # historical study implementation
export PYTHONPATH=ml
PY=.venv/bin/python
$PY -m aqos_ml.run matrix configs/matrix-core.yaml --results results-core
$PY -m aqos_ml.run matrix configs/matrix-aa.yaml --results results-aa
scripts/final-runs.sh   # after the two above: every run after the freeze
$PY -m aqos_ml.run matrix configs/matrix-realcall.yaml \
```

```
    --results results-realcall                    # replayed real calls
AQOS_SERVICE=python AQOS_MODEL=lgbm-v1-fe91bbad $PY -m aqos_ml.run matrix \
    configs/matrix-realcall-v1.yaml --results results-realcall-v1
$PY -m aqos_ml.run matrix configs/matrix-ipv6.yaml --results results-ipv6
$PY -m aqos_ml.compare --dataset data/dataset/v2.jsonl.gz --final
                                      # sealed test, once; needs the data
$PY -m aqos_ml.evidence               # fairness, loss, estimator, dashboard
reports/paper/diagrams/render.sh      # the flowcharts in this report
scripts/demo.sh                       # the scripted demo, about 8 minutes
```

Reproducing the Rust runtime update

The historical commands above explicitly select Go. Rust is the current default; use fresh output paths for new runs so historical results cannot be mistaken for Rust measurements. The following commands regenerate the component comparisons and a six-hour host soak; they do not rerun the original research matrix.

```
unset AQOS_RUNTIME AQOS_SERVICE
make build build-go-reference
.venv/bin/python scripts/bench-rust.py \
    --out reports/rust-classifier-new.json
.venv/bin/python scripts/bench-runtime.py \
    --out reports/rust-extraction-new.json
.venv/bin/python scripts/bench-runtime-controller.py \
    --trials 3 --results reports/rust-controller-new
make soak-runtime SOAK_RESULTS=results-rust-soak-new
```

Keep the host awake during the soak. Its disk guard stops the run if less than 1 GiB remains free. A stopped or interrupted attempt is not a passing result.

Scripted demo

`scripts/demo.sh` runs the whole loop in about eight minutes, without root: it resets the testbed and builds the binaries; introduces the condition by running S6 (a call, then file sharing on the same laptop at 30 s) with plain CAKE and then with AQoSE on the same seed; collects the evidence each run writes (per-packet metrics, CAKE tin statistics, every decision with its reason, the verifier's verdicts, CPU per component); and reports a side-by-side table and a second-by-second chart. `aqos_ml.evidence` computes the fairness, jitter, loss, link-estimation and dashboard figures in this report from finished runs.

Prerequisites

Fedora	43	Python	3.14.7
Linux kernel	7.2.8 (any 7.2.x)	LightGBM	4.7.0
iproute2 (`tc`, `ip`)	6.14.0	scikit-learn	1.9.1
nftables	1.1.3	NumPy, pandas	2.5.3, 3.0.6
Go	1.26.8	matplotlib	3.11.2
Rust	1.99.1	Cargo	1.99.1
`gopkg.in/yaml.v3`	3.0.1	PyYAML, jsonschema, pytest	6.0.3, 4.26.0, 9.1.1

Table 13: Pinned versions (`VERSIONS.md`, `go.mod`, `requirements.lock`). Unprivileged user namespaces must be enabled, as they are by default on Fedora. The report also needs TeX Live, Google Chrome and `pdfcrop` for the flowcharts.

```
make venv     # .venv from requirements.lock
make env      # compares this machine with VERSIONS.md
make build    # Rust runtime and Go testbed into bin/
make check    # Rust, Go reference and Python tests
```

Third-party data and software

Item	Terms	How it is used
ISCX VPN-nonVPN 2016, ISCX Tor/non-Tor 2016 (UNB CIC)	Citation requested	Training and validation; not redistributed
Purdue video-call set 2026	No terms stated; cited	Training and validation; not redistributed
UNSW cloud gaming 2025	No terms stated; cited	Training and validation; not redistributed
Labayen et al. 2020 (Mendeley Data)	CC BY 4.0	Training and validation; not redistributed
VNAT (MIT Lincoln Laboratory) 2020	Cited	Examined; not used for training
Own captures (JioGamesCloud, WhatsApp)	Ours	Training, test, and the two replay traces (sizes and times only)
Linux CAKE, iproute2, nftables	GPL-2.0	Used as installed; not modified
cake-autorate 3.2.2	Its repository's terms	Comparison only; fetched separately, not redistributed
LightGBM; PyYAML, jsonschema, pytest	MIT	Model; configuration, contracts, tests
scikit-learn, NumPy, pandas, SciPy, joblib	BSD-3-Clause	Training, calibration, statistics
matplotlib	Matplotlib licence (PSF-based)	Charts
`gopkg.in/yaml.v3`	MIT and Apache-2.0	Configuration in Go
Mermaid	MIT	Rendering the flowcharts

Frozen system: git commit `4988d91`, model `lgbm-v2-b3f691d8`, dataset v2 (sha256 `b3f691d89f00`). Each run's manifest records the commit the repository was at when it ran, not always `4988d91`: commits made during the final runs changed only reports and documents, apart from the replay generator added for S12 and S13 and the S14 scenario file. Every manifest is also marked `+dirty`, meaning tracked files had uncommitted changes at run time; what those changes were was not recorded. Detailed tables, in `reports/`: `final.md`, `fresh.md`, `realcall.md`, `realcall-v1.md`, `ablation.md` and `head-to-head-v2-test.md`.